Reviewer Pack — Minimal Rank of Quandle Representations and Circuit Monotone…

Entry 09a55af0013d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 17:11:51 UTC

Minimal Rank of Quandle Representations and Circuit Monotone Width Correlation

Entry ID: `09a55af0013d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 17:11:51 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every d-regular graph G, the minimal rank of its associated quandle representation Q(G) is linearly correlated with its circuit monotone width w_m(G), such that min_rank(Q(G)) = Θ(w_m(G)).

Rationale (proposer's reasoning):

> Quandle theory provides a non-commutative algebraic structure that can encode combinatorial properties of graphs. The minimal rank of a quandle representation may capture essential features of the graph's complexity, which could be related to circuit monotone width, a measure of the size of monotone Boolean circuits representing the graph.

Taxonomy category: `Quandle_BooleanCircuits` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `28fc944d017d3a37`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all d-regular graphs G with n vertices (n ≤ 40), the correlation coefficient r between min_rank(Q(G)) and w_m(G) is ≥ 0.8, AND the mean absolute difference between the predicted and actual min_rank(Q(G)) values based on a linear model does not exceed 3 across all 30 seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [0 if i != j else 1 for j in range(n)] for row in matrix]

    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(augmented_matrix[r][i]))
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matr

        if augmented_matrix[i][i] == 0:
            raise ValueError("Singular matrix")

        for j in range(n):
            augmented_matrix[i][j] /= augmented_matrix[i][i]

        for k in range(n):
            if k != i:
                factor = augmented_matrix[k][i]
                for j in range(2 * n):
                    augmented_matrix[k][j] -= factor * augmented_matrix[i][j]

    rank = sum(1 for row in augmented_matrix if any(row[j] != 0 for j in range(n)))
    return rank

def generate_d_regular_graph(d, n):
    graph = {i: [] for i in range(n)}
    edges_added = set()

    while len(edges_added) < d * n // 2:
        u = random.randint(0, n - 1)
        v = random.randint(0, n - 1)

        if u != v and (u, v) not in edges_added and (v, u) not in edges_added:
            graph[u].append(v)
```

```
            graph[v].append(u)
            edges_added.add((u, v))

    return graph

def quandle_representation(graph):
    n = len(graph)
    quandle_matrix = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if j in graph[i]:
                quandle_matrix[i][j] = 1

    return quandle_matrix

def circuit_monotone_width(graph):
    n = len(graph)
    edges = [(i, j) for i in range(n) for j in range(i + 1, n) if j in graph[i]]

    def dfs(node, visited, path):
        if node in visited:
            return False
        visited.add(node)
        path.append(node)

        for neighbor in graph[node]:
            if not dfs(neighbor, visited, path):
                return False

        path.pop()
        visited.remove(node)
        return True

    max_width = 0
    for i in range(n):
        for j in range(i + 1, n):
            if (i, j) in edges:
                continue

            visited = set()
            path = []
            if dfs(j, visited, path):
                max_width = max(max_width, len(path))

    return max_width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    total_min_rank = 0
    total_w_m = 0

    for n in n_values:
        for _ in range(5):
```

```
            graph = generate_d_regular_graph(3, n)
            quandle_matrix = quandle_representation(graph)
            min_rank = gaussian_elimination(quandle_matrix)
            w_m = circuit_monotone_width(graph)

            total_min_rank += min_rank
            total_w_m += w_m
            instances_tested += 1

    mean_min_rank = total_min_rank / instances_tested
    mean_w_m = total_w_m / instances_tested
    correlation_coefficient = (instances_tested * total_min_rank * total_w_m -
                               sum(min_rank * w_m for min_rank, w_m in zip([mean_min_rank] * insta
                              math.sqrt((instances_tested * total_min_rank**2 - sum(min_rank**2 fo
                                         (instances_tested * total_w_m**2 - sum(w_m**2 for w_m in [
    mean_abs_diff = sum(abs(mean_min_rank * w_m / mean_w_m - min_rank) for min_rank, w_m in zip([m

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.8 and mean_abs_diff <= 3,
        "counterexample": "" if correlation_coefficient >= 0.8 and mean_abs_diff <= 3 else "correl
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8 or mean_abs_diff
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df6eff08.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df6eff08.py", line 60, in run_trial
    min_rank = gaussian_elimination(quandle_matrix)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_df6eff08.py", line 20, in gaussian_elim
    augmented_matrix = [row[:] + [0 if i != j else 1 for j in range(n)] for row in matrix]
                                                    ^
UnboundLocalError: cannot access local variable 'i' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17928
2	propose	ollama_remote	glm4:latest	0	0	12743
3	preregistration	ollama_remote	glm4:latest	0	0	16556
4	novelty	ollama_remote	glm4:latest	0	0	57560
5	novelty	ollama_remote	glm4:latest	0	0	9023
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40040
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22237
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20498
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18520
10	judge	ollama_remote	glm4:latest	0	0	15526

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 230631 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/09a55af0013d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/09a55af0013d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/09a55af0013d.tar.gz` (if generated)